GITHUB ADVANCED SECURITY

Detailed Notes - Section-wise, Timestamp-wise, Practical

Built from the provided course outline and notes, with clearly labeled current GitHub documentation additions.

How to use this document: Study each Part in order. First understand the concept, then read the real-world example, then run the YAML/code example mentally, and finally use the triage checklist at the end of the part.

Course Map

Part	Section	Timestamp
Part A	Git and GitHub Foundations	00:00:00 - 00:18:28
Part B	GitHub Advanced Security Overview	00:18:28 - 00:30:13
Part C	Security Overview and Vulnerabilities	00:30:13 - 00:45:51
Part D	Secret Scanning	00:46:56 - 00:57:05
Part E	Dependency Management and Dependabot	01:01:17 - 01:08:39
Part F	Code Scanning	01:20:07 - 01:26:40
Part G	CodeQL Deep Dive	01:38:33 - 01:54:12
Part H	GHAS Best Practices and SDLC	02:17:40 - 02:28:13
Part I	GHAS for Enterprise / Administration	02:29:34 - 02:35:08
Part J	Added Revision Playbook	Supplement

Current terminology note: The course uses GHAS as the umbrella term. Current GitHub documentation splits the paid Advanced Security offering into GitHub Code Security and GitHub Secret Protection. This document keeps the course term GHAS for continuity and flags current terminology where it matters.

PART A - Git and GitHub Foundations

This part is the foundation for understanding GHAS. Security features operate on Git repositories, branches, commits, pull requests, dependencies, and CI/CD workflows, so the Git model must be clear.

00:00:00 - Introduction

The course starts by establishing the Git and GitHub concepts that security automation builds on. Think of Git as version control and GitHub as the collaboration and hosting platform around Git repositories.

> **Real-world example:** A company has 30 developers. Git records every code change, while GitHub hosts the repository and provides pull requests, issues, Actions, security features, and collaboration.

00:09:24 - Git Overview

Git is a free distributed version control system. Its core job is to record source-code history so a team can create versions, compare changes, work in parallel, and roll back when required.

> **Real-world example:** Suppose a release introduces a payment bug. Git allows the team to inspect the commit that introduced the change, revert it, or branch from a stable commit while the bug is fixed.

- Commit = a saved snapshot of repository changes.
- Branch = an independent line of development.
- Checkout/switch = move your working view to another branch or commit.
- Git is distributed: a local clone contains repository history and can work without a constant server connection.

00:10:22 - GitHub Overview

GitHub is a cloud platform for storing and managing Git repositories and for team development. Typical capabilities include repository hosting, pull requests, issues, Projects, Actions, Apps, security tools, and permissions.

> **Real-world example:** A team stores a Java Spring Boot project in a private GitHub repository, reviews pull requests there, runs tests with Actions, and checks Dependabot and CodeQL alerts in the Security area.

00:11:26 - Git Terms

Important terms: repository, working tree, staging area, commit, branch, remote, origin, merge, pull request, fork, tag, and release. A security engineer needs these terms because alerts are attached to repository code, branches, commits, dependencies, or pull requests.

> **Remember:** Do not think of security as a separate system. In GitHub, security signals are connected to the same development objects used by developers.

00:14:53 - GitHub Repository Overview

A repository is the main project container. It can contain source code, configuration, issues, pull requests, Actions workflows, releases, dependency information, and security results.

> **Real-world example:** A repository for an e-commerce API may contain Java code, package files, Dockerfiles, workflow YAML, issue discussions, and the history of security fixes.

00:15:46 - Git Commit Overview

A commit represents a point-in-time change in the repository. Security analysis often associates a finding with a particular file and commit/branch state. Commits also matter for secret scanning because secrets can remain in historical Git data even after deletion from the latest code.

> **Real-world example:** A developer commits an API key, then removes it in a later commit. The current file is clean, but the secret may still exist in history and must be rotated and investigated.

00:16:40 - Git Branch Overview

A branch is a named line of development. Common branches include main, develop, feature branches, and release branches. Security workflows can be triggered or scoped based on branch and pull request events.

> **Real-world example:** A team scans every pull request before merge and performs an additional scheduled scan against the default branch.

00:17:52 - Git Remote, Upstream and Downstream

A remote is a repository reference such as origin. In fork-based contribution models, upstream often means the original project repository and origin often means your fork. Downstream commonly describes work flowing from an upstream source into a consumer or fork.

Example / code

```
git remote -v
git fetch upstream
git switch main
git merge upstream/main
```

Dry run

1. `git remote -v` shows configured remotes.
2. `git fetch upstream` downloads upstream references without changing your branch.
3. You can merge or rebase upstream changes into your local branch.
4. After security fixes in the upstream repository, a downstream fork can synchronize and receive the remediation.

Section Summary

- Git tracks history; GitHub hosts and coordinates collaboration.
- Commits, branches, pull requests, and remotes are the objects GHAS operates around.
- A deleted secret can still exist in Git history, so security scanning must not be limited to the latest file content.

PART B - GitHub Advanced Security Overview

00:18:28 - Advanced Security Overview

GitHub Advanced Security (GHAS) is the course umbrella for GitHub security capabilities that help identify and respond to secrets, vulnerable dependencies, and insecure code. The three course pillars are Secret Scanning, Dependabot, and Code Scanning/CodeQL.

Real-world example: Imagine a Spring Boot application: a leaked database token is handled by secret scanning, a vulnerable Maven package is handled by dependency scanning/Dependabot, and an unsafe SQL construction is found by code scanning/CodeQL.

00:19:08 - GHAS Enabled Plans

The course asks which plans can use Advanced Security. Current GitHub terminology is important: GitHub Code Security covers code scanning and dependency-related advanced features, while GitHub Secret Protection covers secret scanning and push protection. Some security features are free for public repositories; paid Advanced Security products are required for applicable private/internal repository features. [Official docs: current product model]

Remember: For interviews, say “GHAS” when discussing the course umbrella, but know the current product names: GitHub Code Security and GitHub Secret Protection.

00:21:02 - GitHub Security

Security is broader than GHAS. The course lists account security, repository rules management, audit logs, GitHub Security Advisories, and repository policies alongside the GHAS services.

Real-world example: A company can have CodeQL enabled but still be insecure if repository write access is too broad, audit logs are ignored, or security reporting policies are missing.

00:22:56 - GHAS Use Cases

The course connects GHAS to the cost of poor software quality and identifies drivers such as cybercrime, technical debt, and software supply-chain problems. GHAS can reduce risk by detecting secrets, code vulnerabilities, and dependency vulnerabilities earlier in the lifecycle.

Real-world example: If a company has 500 repositories, discovering a leaked token after production deployment is much more expensive than blocking it during a pull request.

00:23:58 - GHAS Components

Secret Scanning detects exposed secrets; Dependabot monitors dependency-related risk; Code Scanning analyzes code for security issues and coding problems. The outputs of all three become alerts that must be triaged and remediated.

Example / code

```
Secret Scanning  -> leaked credentials
Dependabot       -> vulnerable dependency
Code Scanning    -> vulnerable code
```

Dry run

5. A developer pushes a commit containing a cloud token. Secret scanning can create a secret alert or block the push when push protection is enabled.
6. A package version has a known CVE. Dependabot can raise an alert and may offer a security update pull request.
7. A code path contains a SQL injection pattern. CodeQL can create a code scanning alert.

00:25:03 - Taking Action on Alerts

The correct response to an alert is not simply “close it.” First determine whether it is valid, understand impact, identify the owner, fix the vulnerability, and record the remediation or justified dismissal. If the alert is in a public repository you do not own, use the repository owner’s responsible disclosure route.

> **Real-world example:** If a secret alert is valid, rotate/revoke the secret first, remove it from the code/history as appropriate, identify blast radius, and document the incident.

00:26:38 - How to Enable GHAS

The course describes enablement at personal, organization, repository, and plan/administrative levels. In practice, the available control depends on repository visibility, organization ownership, product subscription, and your permissions.

> **Remember:** Separate “feature availability” from “permission to enable” and from “billing/license eligibility.” These are three different questions.

00:27:53 - How to Enable GHAS Automatically

The course highlights organization- and repository-level automatic enablement. Current GitHub also supports security configurations that let organizations apply security enablement settings at scale instead of configuring each repository independently.

> **Real-world example:** An organization creates a “high-security” configuration and applies it to all critical application repositories so secret scanning, push protection, code scanning, dependency features, and other controls are consistent.

00:29:14 - Which GHAS Do You Have?

At the course time, the notes simplify private-repository availability to Dependabot. Current GitHub is more nuanced: dependency graph and some Dependabot features are broadly available, while advanced code and secret protection capabilities depend on the applicable Code Security/Secret Protection product and repository visibility.

> **Remember:** Always check the current organization/enterprise licensing page and product documentation rather than relying on an old plan matrix.

00:30:13 - GHAS Introduction Follow Along

Use this section as a mental setup exercise: open a repository, inspect its Security and quality area, identify which features are enabled, identify the repository visibility, and ask what the organization has purchased and who can access alerts.

Section Summary

- GHAS is a security layer integrated with normal GitHub development workflows.

- The three course pillars are Secret Scanning, Dependabot, and Code Scanning/CodeQL.
- Security features produce alerts, but humans still need to validate, prioritize, fix, or intentionally dismiss them.

PART C - Security Overview and Vulnerabilities

00:41:27 - What Is a Security Vulnerability?

A security vulnerability is a weakness that can be exploited to violate confidentiality, integrity, or availability. The weakness may exist in application code, third-party libraries, configuration, infrastructure, or secret handling.

> **Real-world example:** A web application builds a SQL query directly from user input. An attacker can inject SQL syntax and access data not intended for them.

00:42:41 - Types of Security Vulnerabilities

The course groups examples into web vulnerabilities such as XSS, CSRF, and SSRF; open-source vulnerabilities in libraries; stored secrets; cloud misconfiguration; zero-day vulnerabilities; and malware/ransomware.

> **Real-world example:** A public S3 bucket containing private customer data is a configuration problem even if the application code itself is secure.

00:43:54 - Zero-Day Vulnerabilities

A zero-day is a vulnerability for which defenders have little or no prior warning before exploitation or public disclosure. Detection and remediation are harder because traditional signatures or advisories may not exist yet.

> **Remember:** GHAS reduces many known-risk categories, but it cannot guarantee protection against every unknown vulnerability.

00:44:23 - Most Common Vulnerabilities

For revision, remember four major categories from this course: secrets, dependencies, application code, and configuration. Then map the GitHub service to the category it helps discover.

Example / code

```
Secret in repo       -> Secret Scanning
Vulnerable dependency -> Dependabot / dependency features
Vulnerable code      -> Code Scanning / CodeQL
Config / access      -> GitHub security settings + governance
```

00:44:51 - Finding Vulnerabilities in GitHub

GitHub provides Security and quality views, alerts, dependency information, and security overview capabilities. What you can see depends on repository access and role.

> **Real-world example:** A security manager may view organization-wide security posture, while a developer may only see alerts for repositories they can access. [Current docs: Security overview permissions]

00:45:51 - GitHub Advisory Database

The GitHub Advisory Database contains security advisories for open-source packages and is a major data source for dependency-related alerts. It links affected ecosystems/package versions with vulnerability information and fixes when available.

> **Remember:** Know the difference: CVE is an identification system for vulnerabilities; the GitHub Advisory Database is a repository of advisories used by GitHub security tooling.

Section Summary

- A vulnerability can live in code, dependencies, secrets, configuration, or other layers.
- No single GHAS feature finds everything. Defense in depth is required.
- The GitHub Advisory Database is especially important for dependency vulnerability intelligence.

PART D - Secret Scanning

00:46:56 - Secret Scanning Overview

Secret scanning looks for credentials such as passwords, API keys, personal access tokens, cloud credentials, and other secret patterns in repository content. Its goal is to identify accidental exposure as early as possible.

> **Real-world example:** A developer hard-codes an AWS access key into `application.properties`. Secret scanning recognizes the credential pattern and generates an alert; push protection may prevent the commit from entering the repository.

00:47:31 - Secret Scanning Locations

The course notes say secret scanning can inspect the full Git history across branches and additional collaboration content such as issue data, discussions, and pull requests where supported. The exact coverage depends on the secret-scanning feature and repository configuration.

> **Remember:** The central idea is “do not only scan HEAD.” A secret deleted from the latest file can still exist in historical Git objects.

00:47:58 - Enabling Secret Scanning

Repository administrators can enable secret scanning where the repository and plan support it. At organization scale, administrators can apply consistent security settings across repositories.

Example / code

```
Repository -> Settings -> Security and quality / Advanced Security
          -> Secret scanning
          -> Enable
```

00:48:09 - Secret Scanning Partner Program

GitHub partners with credential providers so partner companies can be notified when one of their known secret formats appears on GitHub. This extends detection beyond your own repository workflow.

> **Real-world example:** A cloud provider recognizes its access-key format. When such a key appears publicly, the provider can participate in the partner program and respond to the exposure.

00:48:47 - Partner Program Use Case

Example: a developer pushes a production API token to a public repository. The secret-scanning partner may detect the token pattern and notify the service provider, which can revoke or invalidate the credential.

> **Remember:** Secret scanning is not a substitute for secret rotation. A valid leaked secret should be treated as compromised until proven otherwise.

00:49:41 - Starting a Scan

A scan may be initiated through enablement/configuration, normal repository activity, or dedicated scanning behavior. The course emphasizes that enabled secret scanning is continuous rather than a one-time action.

> **Real-world example:** Think of it as a security sensor attached to the repository rather than a script you

run once and forget.

00:50:11 - Scan Running Times

The course distinguishes continuous detection from push protection. Push protection scans proposed changes during the push path and prevents secrets from entering the repository when configured. Continuous/background scanning helps find secrets that are already present.

Remember: Push protection is preventative. Continuous scanning is primarily detective. You generally want both.

00:51:08 - Secret Scanning Follow Along

Practice: enable secret scanning in a test repository, intentionally use a safe synthetic test token/pattern, observe the alert, inspect the alert details, and document the remediation path. Never test with real production secrets.

00:53:45 - Set Ignore Follow Along

Ignoring/dismissing a secret alert should be intentional. The reason might be a test fixture, a documented false positive, or another accepted case. The decision should be recorded because ignoring alerts can create security debt.

Real-world example: If a repository contains a clearly fake string used by automated tests, record why the alert is not a real credential rather than blindly ignoring every alert.

00:57:05 - Set Notifications Follow Along

Security notification settings determine who is informed about alerts. Configure them so responsible owners receive actionable information without turning the organization into an alert-noise environment.

Real-world example: A critical repository may notify the application owner and security team immediately, while lower-risk repositories may use team-based notification and daily review.

Practical secret response: 1) Confirm it is a real secret. 2) Revoke/rotate it. 3) Determine where it was exposed. 4) Remove it from the active code path and history where appropriate. 5) Search for additional occurrences. 6) Document the incident and prevention step.

Section Summary

- Secret scanning detects credentials; push protection prevents many new leaks.
- Treat an exposed real secret as compromised and rotate it quickly.
- Use test repositories and synthetic credentials when practicing.

PART E - Dependency Management and Dependabot

01:01:17 - Open Source Popularity

Modern software is built from open-source components because reuse is faster than reinventing everything. But every dependency expands the software supply chain and therefore the attack surface.

> **Real-world example:** A React application may directly use 20 packages and indirectly depend on hundreds of transitive packages.

01:01:42 - Open Source Maintenance Problems

A library can become unmaintained, abandoned, vulnerable, or difficult to upgrade. Even when the package is popular, a vulnerable version may remain in production if teams do not monitor updates.

> **Real-world example:** A logging library has no active maintainer and receives no security patch. A team that depends on it must plan migration to another maintained library.

01:02:19 - Dependency Graph for Open Source

GitHub dependency graph maps dependencies declared by supported manifest and lock files. It helps GitHub understand what packages your repository uses and is a foundation for dependency security features.

> **Real-world example:** For a Maven project, GitHub can reason about the `pom.xml` dependencies; for Node.js, `package.json` and lock files provide important dependency information.

01:02:36 - Dependency Graph Examples

Direct dependency: your project declares it. Transitive dependency: another package brings it in. The security impact can exist at either level.

Example / code

```
Your app
  -> library A (direct)
       -> library B (transitive)
            -> vulnerable library C
```

01:02:59 - Dependabot Overview

Dependabot is GitHub automation for dependency awareness and updates. It can create alerts for vulnerable dependencies supported by GitHub intelligence and can help create update/security-update pull requests where an available fix and ecosystem support exist.

> **Real-world example:** A vulnerable Spring dependency is detected. Dependabot identifies the affected version and may propose an updated version in a pull request.

01:03:19 - Dependabot Use Case Example

Suppose version 1.2.0 of a package has a security advisory and 1.2.3 is patched. Dependabot identifies the vulnerable version and can help move the project toward 1.2.3.

Dry run

8. Read the alert and affected package/version.
9. Confirm the vulnerable path is actually used in your application.

10. Check the suggested patched version and compatibility.
11. Review/test the pull request.
12. Merge only after normal validation succeeds.

01:04:27 - Dependabot Features

Core concepts include dependency graph visibility, Dependabot alerts, update pull requests, security updates, and configuration. Current GitHub also provides advanced auto-triage and dependency review capabilities under the applicable Code Security product.

> **Remember:** Do not confuse “dependency alert” with “automatic fixed code.” The tool detects risk and can propose updates, but the application team remains responsible for validation.

01:05:10 - Enabling Dependabot

Enable the dependency features supported by the repository and organization. In practice, review Settings -> Security and quality / Advanced Security and the dependency graph / Dependabot areas available to your repository.

Example / code

```
# Typical concepts
Dependency graph -> understand what is installed
Dependabot alerts -> understand known vulnerabilities
Dependabot updates -> propose safe version changes
```

01:05:40 - Dependabot Licensing for Private Repositories

The course highlights plan-dependent differences for private repositories. Current product licensing is more nuanced because basic dependency features can be broadly available while premium features are included with GitHub Code Security. Always verify current plan/licensing before making an enterprise design decision.

01:05:58 - Dependabot Private Repository Workflow

A typical flow is: dependency manifest/lock file -> dependency graph -> advisory matching -> alert -> triage -> security update PR or manual upgrade -> tests -> merge -> alert resolved.

> **Real-world example:** A private repository receives a critical dependency alert on Monday. The team patches Tuesday, runs CI Wednesday, and closes the alert after verification.

01:06:23 - Triaging Dependabot Vulnerabilities

Review package name, vulnerable version range, patched version, severity, exploitability context, reachability/use, and whether the dependency is direct or transitive.

> **Real-world example:** If a vulnerable library is present only in a development tool and cannot reach production, risk may differ from the same library used to process untrusted web input.

01:07:32 - Dependabot Limitations

Course notes: not every alert has a security patch; support depends on the dependency graph/ecosystem; archived repositories are not scanned; and detection should not be treated as a guarantee that every malicious package will be identified. Current GitHub now also documents malware alerts for supported

ecosystems, which means the old statement “Dependabot cannot generate malware alerts” is no longer universally true.

> **Remember:** This is an explicit course correction: the older note is outdated. Current GitHub has Dependabot malware alerts for supported ecosystems, with limitations.

01:08:39 - Dependabot Follow Along

Practice with a safe demo repository: add a known vulnerable training dependency, let the graph update, inspect the alert, inspect available remediation, run tests, and close the alert after upgrading.

> **Remember:** Use deliberately vulnerable demo packages only in isolated learning repositories. Never introduce a real vulnerable package into production simply to test the security tooling.

Supplement - Dependency Review - Necessary for SDLC

Dependency review compares dependency changes introduced by a pull request and helps developers understand whether a new or changed dependency introduces known risk before merge. Current GitHub documents dependency review as part of GitHub Code Security.

> **Real-world example:** A pull request adds a new npm package with a critical vulnerability. Dependency review can surface the risk during the review stage before the package becomes part of the default branch.

Section Summary

- Dependency graph answers “what do we depend on?”
- Dependabot answers “is a dependency vulnerable and can we update it?”
- Dependency review answers “what security impact does this pull request introduce?”
- Always validate alerts; not every finding means the same real-world risk.

PART F - Code Scanning

01:20:07 - Code Scanning Overview

Code scanning analyzes source code for vulnerabilities and coding problems. GitHub supports CodeQL and can also ingest results from third-party tools.

> **Real-world example:** A Java service takes user input and passes it into a dangerous command execution function. A static analysis tool identifies the path and raises an alert.

01:20:31 - Supported Repositories

Code scanning availability depends on repository visibility, GitHub product, language/tool support, and permissions. Public repositories have broader free availability; private repositories need applicable Code Security licensing for paid features.

> **Remember:** Never memorize a plan table from an old course. GitHub changes packaging and licensing; verify current docs for enterprise decisions.

01:21:02 - How Code Scanning Works

Conceptually: choose a scanning tool -> run the analyzer -> generate findings -> upload/store results -> show alerts -> triage -> fix or dismiss -> rescan.

Example / code

```
Source code
   |
   v
Analyzer (CodeQL / third party)
   |
   v
Findings (SARIF or native result)
   |
   v
GitHub Code Scanning Alerts
```

01:21:25 - Starting a Scan

A scan can begin when code scanning is enabled, on pushes, pull requests, schedules, or other configured events. Default setup can automatically manage the workflow for CodeQL-supported languages.

> **Real-world example:** A repository runs CodeQL automatically on each push to the default branch and on relevant pull requests.

01:22:44 - Code Scanning Setup Options

The course distinguishes default and advanced/custom setup. Current GitHub recommends default setup as the simplest starting point and advanced setup when you need custom workflow control, special build steps, matrix strategies, or specialized schedules.

> **Remember:** Default setup = low maintenance. Advanced setup = more control, more maintenance.

01:24:16 - Where to Implement Scanning

The scanning point belongs in the SDLC where security feedback is useful. A common model is code -> build -> test -> release -> deploy, with PR and CI security checks preventing known issues from silently moving downstream.

> **Real-world example:** Use pull-request scanning for early feedback and scheduled/default-branch scans for ongoing coverage.

01:25:17 - Code Scanning Actions

Code scanning commonly uses GitHub Actions workflows. An action sets up the analyzer, obtains the code, performs analysis, and uploads results so GitHub can display alerts.

Example / code

```
name: CodeQL
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  schedule:
    - cron: "30 1 * * 1"
jobs:
  analyze:
    runs-on: ubuntu-latest
    permissions:
      security-events: write
      actions: read
      contents: read
    steps:
      - uses: actions/checkout@v4
      - uses: github/codeql-action/init@v3
        with:
          languages: javascript-typescript
      - uses: github/codeql-action/autobuild@v3
      - uses: github/codeql-action/analyze@v3
```

01:25:35 - Scanning Trigger Types

Push, pull request, and scheduled scans are the key course examples. Choose frequency based on risk, cost, and developer workflow; avoid unnecessary scans that create noise without additional value.

> **Remember:** Security automation should be frequent enough to reduce risk but controlled enough to remain actionable.

01:26:40 - Code Scanning Follow Along

Enable default setup on a training repository, inspect the generated configuration, run a scan, open an alert, follow its code path, and observe the next run after remediation.

Section Summary

- Code scanning finds problems in the code itself.
- Default setup is easiest; advanced setup is for customization.
- Code scanning can be native CodeQL analysis or third-party analysis uploaded through SARIF.

PART G - CodeQL Deep Dive

01:38:33 - Enabling Code Scanning

The course begins the CodeQL section by enabling CodeQL analysis. In the GitHub UI, code scanning can be enabled using default setup; advanced setup creates a workflow when deeper control is needed.

> **Real-world example:** Start with default setup for a normal supported project, then move to advanced setup if you need custom builds, custom query packs, special triggers, or matrices.

01:39:00 - Third-Party SARIF Files Overview

SARIF means Static Analysis Results Interchange Format. It is a JSON-based standard used to store static analysis results so GitHub can ingest findings from tools other than CodeQL. [Official docs: SARIF 2.1.0 subset]

> **Real-world example:** A third-party scanner checks your C++ project, emits a SARIF file, and GitHub displays those results as code scanning alerts.

01:39:39 - Uploading Third-Party SARIF Files

SARIF results can be uploaded through GitHub Actions, the code scanning API, or the CodeQL CLI depending on how analysis is produced.

Example / code

```
- name: Upload SARIF
  uses: github/codeql-action/upload-sarif@v3
  with:
    sarif_file: results.sarif
```

01:40:09 - Third-Party SARIF File Example

A SARIF record typically identifies the rule, message, artifact/file location, and other metadata. GitHub converts the imported result into a code scanning alert experience.

Example / code

```
{
  "version": "2.1.0",
  "runs": [{
    "tool": {"driver": {"name": "ExampleScanner"}},
    "results": [{
      "ruleId": "SQL001",
      "message": {"text": "Possible SQL injection"},
      "locations": [{"physicalLocation": {"artifactLocation": {"uri": "src/db.js"}, "region":
{"startLine": 42}}}]
    }]
  }]
}
```

01:40:42 - Default Code Scanning

Default setup automatically creates and manages the CodeQL configuration based on detected CodeQL-supported languages. Current docs describe it as the easiest, lowest-maintenance approach.

> **Remember:** Use default setup first unless there is a concrete requirement for advanced configuration.

01:41:36 - Custom Code Scanning

Advanced setup lets you control languages, triggers, query packs/suites, build steps, runners, and workflow structure. This is useful for compiled languages or non-standard builds.

> **Real-world example:** A monorepo contains Java and TypeScript plus a custom generated-code build step. Advanced setup can give the workflow precise control over how the analysis is built and executed.

01:41:58 - Code Language Detection Scanning

Default setup can detect which supported languages are present and adjust analysis configuration. Current docs note that the setup can update as repository languages change.

> **Remember:** Language support is analyzer-specific. “GitHub can scan code” does not mean every language has identical CodeQL coverage.

01:42:34 - CodeQL Database Analysis

CodeQL transforms source code into a database that captures program structure and relationships. The database can then be queried like data rather than merely scanning text patterns.

> **Real-world example:** Instead of asking “does this file contain string X?”, CodeQL can reason about relationships such as input flowing into a sensitive sink.

01:43:15 - CodeQL Query Analysis

Queries inspect the CodeQL database and return locations/patterns that meet defined conditions. Security queries are designed to identify vulnerability patterns.

> **Real-world example:** A query can track untrusted input reaching a dangerous function without validation.

01:44:10 - Types of CodeQL Queries

Queries can be used for security vulnerabilities, code quality/maintainability, and custom organization-specific rules. The exact query set depends on the selected suite/pack and language.

> **Real-world example:** A company could write a custom query to detect a forbidden cryptographic library or internal insecure helper.

01:44:52 - CodeQL Queries Deep Dive

The course example searches Ruby `if` expressions that contain no actionable body. This illustrates how CodeQL reasons about an abstract syntax tree rather than raw text.

Example / code

```
import codeql.ruby.AST

from IfExpr ifexpr
where not exists(ifexpr.getThen())
select ifexpr, 'This if expression is redundant.'
```

Dry run

13. `import` loads the Ruby AST library.
14. `from` introduces the variables being analyzed (`ifexpr`).

15. `where` filters those variables using a logical condition.
16. `select` emits the matching code element and a message.
17. The result is a custom finding for redundant `if` expressions.

01:45:42 - Code Query Anatomy

The course lists metadata, import, from, where, and select. In practice, query metadata identifies the query and how GitHub should treat it, imports provide libraries, `from` defines variables, `where` defines conditions, and `select` defines the result.

Example / code

```
@name My Security Query
@description Finds an unsafe pattern
@kind problem
@problem.severity warning
@precision high
@id custom/unsafe-pattern
@tags security

import <language-library>
from <type> x
where <condition>
select x, "Explain the finding"
```

01:47:29 - Code Query Suite

A query suite is a collection/selection of queries that can be run together rather than specifying each query file one by one. Built-in suites and custom suites help balance coverage and precision.

Real-world example: A team can choose a standard suite for normal repositories and a broader suite for sensitive applications.

01:47:52 - Types of Code Query Suites

Current built-in suites are `default`, `security-extended`, and `security-and-quality` where supported. The course specifically contrasts default and security-extended.

Real-world example: Default prioritizes high-value, higher-precision security queries; security-extended adds additional queries with somewhat lower precision and potentially more false positives. `security-and-quality` adds maintainability/reliability-oriented checks and is available for advanced setup. [Official docs]

01:48:28 - Code Query Findings

Query findings become code scanning alerts. Each alert must be triaged: confirm the issue, understand exploitability, fix or justify dismissal, and rescan.

Real-world example: A high-severity alert on an internet-facing endpoint receives higher priority than a low-severity maintainability warning in test-only code.

01:49:04 - CodeQL Packs

CodeQL packs package queries, libraries, and related metadata so analysis content can be shared and reused. Current GitHub docs describe query packs, library packs, and model packs.

Real-world example: An organization publishes an internal query pack with rules for company-specific

authentication practices and reuses it across many repositories.

01:50:08 - Code Scanning Workflow

The normal workflow is configure analyzer -> checkout/build -> analyze -> upload results -> review alerts -> fix -> rescan. The analyzer can be CodeQL or a third-party tool that produces compatible results.

Example / code

```
PR / push
   |
   v
CodeQL / third-party scanner
   |
   v
Results -> GitHub
   |
   v
Triage -> Fix / Dismiss -> Rescan
```

01:50:50 - CodeQL Partner Integrations

The course refers to partner integrations and the GitHub registry of CodeQL packs. The core idea is that CodeQL analysis can be extended with reusable query content and integrated into broader security workflows.

Remember: Treat third-party query packs and scanners as software dependencies themselves: review source, ownership, update cadence, and trust.

01:51:25 - Workflow Priority Order

When multiple code scanning configurations or tools exist, results may have separate analysis origins. Avoid configuring overlapping workflows accidentally because duplication creates noise and operational cost.

Real-world example: A repository accidentally runs two CodeQL workflows on every push; identical alerts appear from multiple origins and confuse triage. Consolidate the workflows.

01:51:55 - Alerts Workflow

Course model: scan -> alerts -> triage -> remediation -> closure. Modern GitHub also lets teams dismiss alerts with reasons and optional comments for auditability.

Example / code

```
Alert raised
   -> Is it real?
      -> Yes: assess impact -> fix -> test -> rescan
      -> No: dismiss with documented reason
   -> Track recurrence / root cause
```

01:53:08 - Alerts Security Incident Example

Imagine a CodeQL alert identifies command injection in a file upload endpoint. The security team validates exploitability, assigns the owner, raises priority based on exposure, and tracks remediation until the scanner no longer reports the issue.

Real-world example: If the endpoint is internet-facing and accepts attacker-controlled input, prioritize it over a similar alert in an internal test tool.

01:54:12 - CodeQL Follow Along

Practice: enable CodeQL on a training repository, inspect an alert, open the source location, understand the data flow/path explanation where available, fix the issue, rerun analysis, and verify the alert closes.

Section Summary

- CodeQL is a semantic analysis engine, not just a text grep tool.
- Database + query = reusable analysis model.
- Query suites control breadth/precision; packs package and share query content.
- SARIF lets third-party tools use GitHub's code scanning alert experience.

PART H - GHAS Best Practices and SDLC

02:17:40 - Software Development Lifecycle (SDLC)

The SDLC describes how software moves from idea and configuration through coding, testing, release, deployment, and operation. Security controls should be inserted at multiple points rather than waiting for the final release.

Example / code

```
Project inception
   -> Project configuration / security policy
   -> Code / test / merge
      -> Code scanning
      -> Secret scanning
      -> Dependency review / Dependabot
   -> CI testing
   -> CD
   -> QA / integration testing
   -> Ship / operate
```

02:18:30 - SDLC Restricting Access

Security results contain sensitive information about weaknesses in applications. Access should follow least privilege: people should get enough access to perform their job, not unrestricted visibility into every repository and alert.

Real-world example: A developer works on one payment service and does not automatically need read access to security alerts for every internal repository.

02:19:11 - SDLC Security Overview

Security overview gives authorized users a higher-level picture of repository risk. Current GitHub documentation states that the visible scope depends on role and permissions.

Real-world example: A security manager may review organization-level vulnerability distribution while a developer sees only repositories they can access.

02:19:54 - SDLC Security Policies

A repository security policy explains how contributors should report security vulnerabilities responsibly. The course describes using a `SECURITY.md` file in `.github`.

Example / code

```
# .github/SECURITY.md

## Reporting a vulnerability
Please report suspected vulnerabilities privately.
Do not open a public issue containing sensitive details.
Provide reproduction steps, affected versions, and impact when safe to do so.
```

02:20:28 - SDLC Secret Scanning

Secret scanning belongs both in repository monitoring and in the developer workflow. Push protection is especially valuable because it catches a leak before it becomes part of normal repository history.

Real-world example: A developer pastes a cloud token into a configuration file during a feature branch

commit. Push protection blocks the push and guides them toward remediation.

02:21:10 - SDLC Security Workflow

A practical workflow is: policy -> detect -> triage -> remediate -> verify -> learn. Security should be treated as a normal engineering process rather than a one-time audit.

Remember: The goal is not zero alerts forever. The goal is controlled risk, fast remediation, and prevention of repeated root causes.

02:22:02 - Types of Vulnerabilities

Use three identifiers in your vocabulary: CVE for individual known vulnerabilities, CWE for weakness classes, and severity/risk rating for prioritization. A CVE can map to one or more CWE categories.

Real-world example: A SQL injection CVE may be classified under the broader CWE category for improper neutralization of special elements in SQL commands.

02:23:16 - GitHub Advisory Database

Use the advisory database to connect dependency versions with known public vulnerability intelligence. Dependency tooling uses this information to alert developers when an installed version is affected.

Remember: An advisory is evidence that a dependency version is known to be risky; application context still matters for prioritization.

02:24:03 - Developer Roles and Responsibilities

Developers should keep dependencies current, avoid committing secrets, respond to security alerts, review security findings introduced by their changes, and write secure code.

Real-world example: When a CodeQL alert is assigned to your pull request, your responsibility is to understand and remediate it, not simply move it out of sight.

02:25:09 - Security Roles and Responsibilities

Security/application-security roles help define policy, threat models, triage standards, escalation paths, governance, and organization-wide visibility. Security teams should not become the only group fixing every finding; developers need ownership.

Remember: Security team = governance and expertise. Development team = code ownership and remediation. Both must collaborate.

02:25:37 - Admin Roles and Responsibilities

Administrators manage access, repository organization, security settings, licensing, policies, and operational support. They are responsible for enabling the platform correctly, not for deciding every application-level fix.

Real-world example: An organization administrator enables required security settings and team access, while the application team fixes a vulnerable controller.

02:26:13 - Additional Roles and Responsibilities

Depending on organization design, you may have repository owners, project owners, platform engineering, DevOps, incident responders, compliance, and legal/privacy teams. Define handoffs before a security incident occurs.

> **Real-world example:** A secret leak can require developer remediation, security investigation, platform log review, and business/legal notification depending on impact.

02:27:11 - Notifying Responsible Parties

Notification should reach the person/team that can act. Use severity, exploitability, asset criticality, data sensitivity, and exposure to determine escalation.

> **Real-world example:** A critical vulnerability in a public payment API should trigger faster escalation than a low-risk issue in an internal demo repository.

02:28:13 - Triage Workflow Based on Risk Ratings

Course notes include Critical, High, Medium, Low plus Error, Warning, and Note for code scanning. Current GitHub documentation distinguishes generic result level (`Error`, `Warning`, `Note`) from security severity (`Critical`, `High`, `Medium`, `Low`).

Example / code

```
Priority = exploitability + business impact + exposure + asset criticality

Example:
Critical + internet-facing + sensitive data -> immediate escalation
Low + test-only + no production path -> lower priority / possible dismissal with evidence
```

> **Golden triage sequence:** Validate -> classify -> assign -> remediate -> verify -> document. Never send hundreds of raw findings to developers without a basic validity/triage filter.

Section Summary

- Security must be embedded across the lifecycle.
- Use least privilege for security data and administrative actions.
- Roles should be explicit: developers own code fixes; security owns governance/expertise; admins own platform access/configuration.
- Triage is a process, not a notification flood.

PART I - GHAS for Enterprise / Administration

02:29:34 - Should I Purchase a GHAS License?

The course asks whether the security capabilities justify licensing. The business decision should consider repository count, sensitivity, developer count/committers, supply-chain exposure, regulatory obligations, current tooling, desired automation, and the cost of untreated vulnerabilities.

Real-world example: An enterprise with hundreds of private repositories and a large open-source dependency footprint may gain more value from centrally managed secret/code/dependency security than from ad-hoc scripts.

02:30:08 - Enabling GHAS in GitHub

GHAS-related features can be configured at repository and organization/enterprise scope depending on the feature and plan. Current GitHub supports organization-level security configurations to enable security features at scale.

Real-world example: A platform team defines one baseline configuration, applies it to critical repositories, monitors coverage, and rolls out broader enablement gradually.

02:30:49 - Levels of Enablement

Think in layers: individual repository configuration, organization-level defaults/configurations, and enterprise governance. Centralized configuration provides consistency; repository-level customization handles special cases.

Remember: Central policy should define the minimum baseline; repositories can add stricter controls when needed.

02:31:42 - Levels of Access to Alerts

Alert visibility depends on repository permission and organization/enterprise role. Current security overview documentation notes that organization owners and security managers have broad organization-level visibility, while other members are limited by repository access.

Real-world example: A security manager can review risk across repositories, while a contributor may see only repositories where they have access to security alerts.

02:33:16 - Required Level of Access

The exact permission depends on the alert type and action. Common concepts are repository write/admin access, organization ownership, security manager role, and appropriate security-event permissions for automation workflows.

Remember: When troubleshooting “I cannot see/fix an alert,” check both repository access and security-specific role/permissions.

02:33:52 - Security Overview

Security overview aggregates security information so teams can identify where the largest risks are. Current GitHub docs emphasize that the data shown is constrained by the viewer’s role and permissions.

Real-world example: Instead of opening 300 repositories one by one, a security manager can identify which repositories have the highest volume of critical or unresolved security issues.

02:34:27 - GHAS Logging

Logging and auditing help answer “who changed what and when?” The course connects GHAS operations with GitHub audit logging and security governance. For enterprise use, review audit logs for security-setting changes, permission changes, workflow-related actions, and other significant administrative events.

Remember: Security controls without auditability are difficult to govern.

02:35:08 - API Endpoints for Security

GitHub exposes APIs for many security operations, including code scanning, Dependabot alerts, secret scanning alerts, and other repository/security data. APIs allow reporting, integrations, dashboards, and automation.

Example / code

```
# Example idea (REST API shape, not a live command)
GET /repos/{owner}/{repo}/code-scanning/alerts
GET /repos/{owner}/{repo}/dependabot/alerts
GET /repos/{owner}/{repo}/secret-scanning/alerts
```

Remember: API availability and endpoint details evolve. Use current GitHub REST/GraphQL documentation for exact paths, permissions, and headers before implementing automation.

Section Summary

- Enterprise security is primarily a governance and scale problem.
- Use centralized enablement, least-privilege access, and auditability.
- APIs turn GHAS data into dashboards, ticketing integrations, and automated workflows.

PART J - Practical Revision Playbook (Added)

This section is added to make the course easier to revise and apply in real projects. It does not replace the source material; it organizes it into repeatable engineering decisions.

J.1 - GHAS Mental Model

Memorize the four-step security loop: Prevent -> Detect -> Triage -> Remediate.

Example / code

```
Prevent: Push Protection / policy
Detect: Secret Scanning / Dependabot / Code Scanning
Triage: validate + prioritize + assign
Remediate: rotate / upgrade / fix / verify
```

J.2 - Which Tool Finds What?

Use this mapping during interviews and real work.

Example / code

```
Leaked API key / token       -> Secret Scanning
Known vulnerable dependency  -> Dependabot / dependency features
Insecure application logic   -> Code Scanning / CodeQL
Dependency added in a PR     -> Dependency Review
Security governance          -> Security Overview / Policies / Audit Logs
```

J.3 - Alert Triage Example - Secret

Scenario: a production cloud token appears in a public repository. Correct response: stop assuming it is harmless, revoke/rotate the credential, identify usage, assess exposure, remove it, and record the incident.

Dry run

18. Validate the token pattern and provider.
19. Revoke/rotate immediately.
20. Search repository history and other repositories for copies.
21. Check cloud logs for suspicious use.
22. Remove the secret from code/configuration.
23. Add preventive controls such as push protection and secret management.

J.4 - Alert Triage Example - Dependency

Scenario: a critical Maven dependency vulnerability is found. Do not stop at “Dependabot found it.” Determine affected versions, patched version, production reachability, exploitability, and rollback/upgrade risk.

Dry run

24. Confirm package/version and advisory.
25. Check whether the vulnerable component is actually included at runtime.
26. Upgrade to a fixed version where possible.
27. Run unit/integration/security tests.
28. Deploy safely and monitor.
29. Close the alert only after evidence of remediation.

J.5 - Alert Triage Example - CodeQL

Scenario: CodeQL reports command injection in a controller. The team traces untrusted input to the command sink, introduces validation or safer APIs, adds regression tests, and rescans.

Example / code

```
Bad pattern (conceptual):
exec(userInput)

Better design:
- avoid shell execution
- use allowlisted commands/parameters
- validate input
- use safe APIs
```

J.6 - When to Use Default vs Advanced CodeQL Setup

Use default setup when you want quick, low-maintenance coverage for supported languages. Use advanced setup when you need custom builds, custom query suites/packs, matrices, schedules, runners, or specialized workflow behavior.

> **Remember:** The best setup is not the most complicated setup. Start simple, then add complexity only when a real requirement appears.

J.7 - Current Product Model - 2026 Reference

Current GitHub documentation describes two paid Advanced Security products: GitHub Code Security and GitHub Secret Protection. Code Security includes code scanning, premium Dependabot features, dependency review, security overview, CodeQL CLI, and related capabilities. Secret Protection includes secret scanning and push protection. Some features are available free for public repositories; private/internal usage may require the relevant product.

> **Remember:** This is a current documentation update to older GHAS course material. Use the current product names when discussing licensing with a modern GitHub organization.

J.8 - What Is a Good Enterprise Rollout?

A practical rollout is staged: discover -> baseline -> pilot -> measure -> enforce -> expand.

Dry run

30. Discover current repository inventory, languages, secrets, dependencies, and owners.
31. Baseline security policies and required permissions.
32. Pilot security configurations on high-value repositories.
33. Measure alert volume, remediation time, false-positive rate, and coverage.
34. Tune query suites, notifications, and ownership.
35. Expand to the rest of the organization and enforce minimum standards where appropriate.

J.9 - Interview Questions You Should Be Able to Answer

Use these as oral revision prompts: What is GHAS? What are its pillars? What is the difference between secret scanning and push protection? What is a dependency graph? What does Dependabot do? What is CodeQL? What is SARIF? What is a CodeQL database? What is a query suite? What is a query pack? Default vs advanced setup? How do you triage an alert? What is least privilege? What is a security policy? How does SDLC integrate security?

Remember: Try answering each question in 30-60 seconds before reading the explanation again.

Glossary

Term	Meaning
CVE	Common Vulnerabilities and Exposures - identifier for a specific publicly disclosed vulnerability.
CWE	Common Weakness Enumeration - classification of a weakness pattern, such as SQL injection or improper input validation.
CVSS	Common Vulnerability Scoring System - standardized scoring used to describe vulnerability severity.
GHAS	Course umbrella term for GitHub Advanced Security capabilities.
GitHub Code Security	Current GitHub paid security product covering code scanning and dependency-related advanced features.
GitHub Secret Protection	Current GitHub paid security product covering secret scanning and push protection.
CodeQL	GitHub semantic code analysis engine used for code scanning.
SARIF	Static Analysis Results Interchange Format, a JSON-based format for static-analysis results.
Dependency Graph	GitHub representation of the dependencies used by a repository.
Dependabot	GitHub automation for dependency alerts and dependency updates.
Push Protection	Preventive secret detection during push so many secrets are blocked before they enter the repository.
Security Overview	Organization/enterprise security posture view whose visibility depends on role and repository access.
Security Policy	Repository guidance for responsible vulnerability reporting, commonly stored in `.github/SECURITY.md`.
Security Manager	GitHub organization role with broad security-management capabilities where supported.
OIDC	OpenID Connect mechanism used by CI/CD to

obtain short-lived cloud credentials without storing long-lived cloud secrets in GitHub.

Official References Used for Current-Feature Verification

Reference	URL
1. About GitHub Advanced Security	https://docs.github.com/en/get-started/learning-about-github/about-github-advanced-security
2. GitHub Advanced Security license billing	https://docs.github.com/en/billing/concepts/product-billing/github-advanced-security
3. Security overview	https://docs.github.com/en/code-security/concepts/security-at-scale/security-overview
4. About setup types for code scanning	https://docs.github.com/en/code-security/concepts/code-scanning/setup-types
5. About SARIF files for code scanning	https://docs.github.com/en/code-security/concepts/code-scanning/sarif-files
6. CodeQL query suites	https://docs.github.com/en/code-security/concepts/code-scanning/codeql/codeql-query-suites
7. CodeQL query packs	https://docs.github.com/en/code-security/concepts/code-scanning/codeql/query-packs
8. Configuring default setup for code scanning	https://docs.github.com/en/code-security/how-tos/find-and-fix-code-vulnerabilities/configure-code-scanning/configure-code-scanning
9. Resolving code scanning alerts	https://docs.github.com/en/code-security/how-tos/manage-security-alerts/manage-code-scanning-alerts/resolve-alerts
10. Buying Advanced Security for your organization or enterprise	https://docs.github.com/en/billing/how-tos/products/buy-advanced-security
11. Dependabot malware alerts	https://docs.github.com/en/code-security/concepts/supply-chain-security/malware-alerts

One-Page Final Revision Checklist

- Can I explain Git vs GitHub in one minute?
- Can I map Secret Scanning, Dependabot, Dependency Review, and Code Scanning to the problem each solves?
- Can I explain prevent vs detect using Push Protection and continuous scanning?
- Can I explain dependency graph, direct vs transitive dependency, and Dependabot workflow?
- Can I explain CodeQL database -> query -> alert?
- Can I explain SARIF and why third-party tools use it?
- Can I distinguish default vs advanced CodeQL setup?
- Can I explain default vs security-extended query suites?
- Can I distinguish query suites from query packs?

- Can I describe an alert triage workflow without sending every alert directly to developers?
- Can I explain CVE vs CWE vs CVSS?
- Can I explain the role of a security policy / SECURITY.md?
- Can I explain least privilege and who should access security alerts?
- Can I explain the current GitHub Code Security vs GitHub Secret Protection product model?
- Can I describe how GHAS fits into the SDLC from commit to production?